

《构建 Cesium 三维地球场景》实验指导书

实验目标

通过本实验，学生将掌握以下技能：

- 安装、配置并使用 Cesium 进行三维地球场景的可视化。
- 控制三维地球的 UI 控件。
- 三维场景中添加立方体要素

实验环境

- ◆ 编程语言：JavaScript
- ◆ 开发工具：Visual Studio Code 或其他支持 JavaScript 的 IDE
- ◆ Cesium 是一个用于创建 3D 地球和地图可视化应用的开源 JavaScript 库。它基于 WebGL 技术，可以在现代浏览器中实现高性能的地理空间可视化效果。Cesium 支持多种数据格式（如 GeoJSON、KML、3D Tiles 等），并且可以轻松集成到 Web 应用中。
- ◆ Cesium 的核心功能包括：

创建三维地球场景。

支持高精度地形和影像图层。

提供丰富的实体（Entity）API 用于图形绘制。

支持时间动态数据的可视化。

可扩展性强，支持插件和自定义渲染。

实验步骤

步骤 1：安装依赖库，下载 Cesium JS 库

步骤 2：配置 Cesium 项目

创建一个新的项目文件夹，并在该文件夹中解压缩 cesium 的文件夹

引入 Cesium 资源

使用 `<script>` 引入了 Cesium.js 库。

使用 CSS 导入了 Cesium 的默认控件样式文件：widgets.css。

设置容器

`<div id="cesiumContainer">` 是 Cesium 渲染 3D 场景的容器。

初始化 Viewer

使用 Cesium.Viewer 初始化了一个 3D 地球场景。

设置了 Bing Maps 的访问令牌 `bingMapAccessToken`，用于加载底图数据。

禁用了部分默认控件（如缩放按钮、全屏按钮等），以简化界面。

步骤 3：核心代码解析

HTML 代码

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Hello World!</title>
  <script src="../Build/Cesium/Cesium.js"></script>
  <style>
    @import url(../Build/Cesium/Widgets/widgets.css);
    html, body, #cesiumContainer {
      width: 100%; height: 100%; margin: 0; padding: 0; overflow: hidden;
    }
  </style>
</head>
<body>
  <div id="cesiumContainer"></div>
  <script>
    // JavaScript 代码
  </script>
</body>
</html>

```

JavaScript 部分

```

var viewer = new `Cesium.Viewer`('cesiumContainer', {
  geocoder: false,
  homeButton: false,
  sceneModePicker: false,
  baseLayerPicker: false,
  navigationHelpButton: false,
  animation: false,
  timeline: false,
  fullscreenButton: false,
  vrButton: false
});

```

注释掉的实体添加代码

```

javascript
// var redBox = viewer.entities.add({
//   name : 'Red box with black outline',
//   position: Cesium.Cartesian3.fromDegrees(-107.0, 40.0, 300000.0),
//   box : {
//     dimensions : new Cesium.Cartesian3(400000.0, 300000.0, 500000.0),
//     material : Cesium.Color.RED.withAlpha(0.5),

```

```
//      outline : true,  
//      outlineColor : Cesium.Color.BLACK  
//    }  
// });  
// viewer.zoomTo(viewer.entities);
```

这段代码被注释掉了，但其功能是向地球场景中添加一个红色盒子，并将视角缩放到该盒子的位置。

步骤 4 程序运行步骤：

4.1 环境准备

安装 Node.js 和 npm；

下载 Cesium 并解压；

在项目目录下启动本地服务器（例如使用 HTTP Server）：

```
node server.js
```

4.2 运行流程

打开浏览器，访问 <http://localhost:8080/HelloWorld.html>；

页面加载后，会显示一个 3D 地球场景；

默认界面上已移除了大部分控件；

如果取消注释添加红盒子相关代码，页面还会显示一个半透明的红色立方体。

实验总结

一、学习收获

通过本次实习，掌握了以下内容：

1. 如何在 HTML 中引入并使用 Cesium 库；
2. 了解了 Cesium.Viewer 的基本配置；
3. 掌握了如何通过 Entity API 添加 3D 模型或几何图形；
4. 熟悉了如何控制场景中的 UI 组件；
5. 初步了解了 WebGIS 在 Web 端的展示方式。

二、改进建议

1. 可以尝试加载更多数据类型（如 GeoJSON、KML 等）来丰富地球场景；
2. 增加交互功能，比如点击事件、动画播放等；
3. 加入时间轴控件，实现历史数据的动态回放；
4. 集成天气、光照等更复杂的仿真场景。

注意事项

- MongoDB 服务运行：确保 MongoDB 服务已正确安装并运行，否则无法连接数据库。
- 数据格式匹配：确保 MongoDB 中的数据格式与 Mongoose 模型定义匹配，避免查询时出现错误。
- Cesium 配置：确保 Cesium 的静态资源正确加载，避免页面显示问题
- Cesium 账号：确保注册 Cesium 的 ion 账户，否则无法正确加载地图

《Cesium 三维地球添加点云、模型》实验指导书

实验目标

- 掌握 Cesium 中加载 3D Tiles 点云数据的基本方法。
- 掌握 Cesium 中加载 3D Tiles 模型数据的方法。
- 学习通过按钮交互控制模型旋转、相机视角等操作。
- 理解 Cesium 中实体（Entity）和图元（Primitive）的简单使用。

实验环境

Cesium 库（版本 1.xx 以上）

本地 Web 服务器（Node.js）

现代浏览器（Chrome / Firefox / Edge）

数据文件：点云 tileset.json、模型 Tileset/tileset.json、底图图片 ../img/worldimage.jpg

实验步骤

实验 1：加载点云数据并设置样式

步骤：

创建 HTML 文件，引入 Cesium 核心库与样式。

初始化 Viewer，禁用不需要的控件（如 homeButton、timeline 等），并设置自定义底图（单张图片）。

使用 viewer.scene.primitives.add 加载点云 3D Tiles 数据。

为点云设置样式：统一颜色为红色，点大小为 5.0。

使用 flyTo 方法将相机飞行到指定经纬度位置。

核心代码：

javascript

```
var viewer = new Cesium.Viewer('cesiumContainer', {
    imageryProvider: new Cesium.SingleTileImageryProvider({ url: '../img/worldimage.jpg' }),
    homeButton: false,
    sceneModePicker: false,
    baseLayerPicker: false,
    navigationHelpButton: false,
    animation: false,
    timeline: false,
    fullscreenButton: false,
```

```

        vrButton: false
    });

    var tileset = viewer.scene.primitives.add(new Cesium.Cesium3DTileset({ url:
    'pwt1/pts/tileset.json' }));
    tileset.style = new Cesium.Cesium3DTileStyle({
        color: "color('#ff0000')",
        pointSize: 5.0
    });

    viewer.camera.flyTo({
        destination: Cesium.Cartesian3.fromDegrees(112.82536, 23.071506, 500.0),
        orientation: { heading: Cesium.Math.toRadians(145.0), pitch: Cesium.Math.toRadians(-35.0),
        roll: 0.0 }
    });

```

实验 2：添加圆形区域（椭圆）

步骤：

使用 `viewer.entities.add` 添加两个圆形实体，指定位置、半轴长度和材质颜色。

代码示例：

```

javascript
viewer.entities.add({
    position: Cesium.Cartesian3.fromDegrees(112.82536, 23.071506, 100),
    ellipse: {
        semiMinorAxis: 300.0,
        semiMajorAxis: 300.0,
        material: Cesium.Color.BLUE.withAlpha(1.0)
    }
});

```

实验 3：加载 3D Tiles 模型并实现旋转交互

步骤：

与实验 1 类似初始化 `Viewer`，加载模型 `Tileset/tileset.json`。

在模型 `readyPromise` 中调整相机视角，使模型居中显示。

在页面中添加三个按钮，分别控制模型绕 X、Y、Z 轴旋转。

编写旋转函数，每次点击增加旋转角度，更新模型的 `modelMatrix`。

核心代码：

```
javascript
var tileset = viewer.scene.primitives.add(new Cesium.Cesium3DTileset({ url:
'Tileset/tileset.json' }));
var m = tileset.modelMatrix; // 保存原始矩阵

function rotatex() {
    anglex += 1;
    var rotMat = Cesium.Matrix3.fromRotationX(Cesium.Math.toRadians(anglex));
    tileset.modelMatrix = Cesium.Matrix4.multiplyByMatrix3(m, rotMat, new
Cesium.Matrix4());
}
// 类似实现 rotatey、rotatez
说明： 旋转是累积的，每次在原矩阵基础上乘以新的旋转矩阵。
```

交互功能说明

操作 功能

页面加载 自动加载点云、模型、圆形区域，相机飞至指定位置

按钮“rotatex” 模型绕 X 轴旋转 1 度（累积）

按钮“rotatey” 模型绕 Y 轴旋转 1 度（累积）

按钮“rotatez” 模型绕 Z 轴旋转 1 度（累积）

注意事项

Cesium 库路径错误

确保 Cesium.js 和 widgets.css 路径正确，建议使用绝对路径或相对路径验证。

3D Tiles 数据无法加载

检查 tileset.json 及相关瓦片文件是否在指定目录下。

使用浏览器开发者工具查看网络请求，确认文件是否成功获取。

注意跨域问题：本地服务器需配置 CORS。

模型旋转不生效

确保 tileset.modelMatrix 已正确获取初始矩阵。

旋转角度需转换为弧度：Cesium.Math.toRadians(anglex)。

点云样式不显示

检查点云数据本身是否有点大小属性，若没有，pointSize 可能无效，可尝试使用 pointSize:

5.0 强制设置。